

Cloud Integration and Remote Monitoring System

1. Project Overview

The **Cloud Integration and Remote Monitoring System** is an IoT-based extension of the Worker Health and Safety Monitoring System. Its main purpose is to collect information from the worker-monitoring device, send the data to a cloud platform through the **ESP8266 NodeMCU**, store the information, and make it available to supervisors through a web or mobile dashboard.

The proposed system uses the **Pulse Sensor, DHT11, MQ-2 Gas Sensor, OLED Display, Buzzer, and NEO-6M GPS**. The ESP8266 acts as the central controller and Wi-Fi communication device.

The system continuously collects sensor information and sends it to the cloud. A supervisor can remotely monitor worker status, environmental conditions, gas alerts, and GPS location.

Main objectives

- Provide real-time remote worker monitoring.
- Upload sensor data to the cloud automatically.
- Store historical sensor readings.
- Monitor workplace environmental conditions.
- Detect gas/smoke-related hazards.
- Monitor pulse activity.
- Track worker location using GPS.
- Generate immediate local safety alerts.
- Provide remote alerts to supervisors.
- Display information through a cloud dashboard.

Major advantages

- Real-time monitoring
- Remote accessibility
- Automatic data storage
- IoT-based communication
- Emergency alert generation
- GPS-based location monitoring

- Historical data analysis
 - Expandable architecture
-

2. Problem Statement

Workers in construction sites, factories, mines, warehouses, and other hazardous environments may be exposed to dangerous conditions. Continuous manual monitoring of every worker is difficult, especially when workers operate in large or isolated areas.

Traditional monitoring systems may provide only local information. If a worker experiences an abnormal condition, encounters dangerous gas, or requires assistance in a remote location, the supervisor may not receive the information immediately.

Another problem is the lack of centralized historical data. Without stored information, it is difficult to analyze previous incidents, identify recurring hazards, and evaluate workplace safety.

Therefore, an IoT-based **Cloud Integration and Remote Monitoring System** is required to provide continuous data collection, cloud storage, remote visualization, and rapid notification of safety events.

Problems addressed

1. Lack of remote worker monitoring.
 2. Delayed emergency communication.
 3. Difficulty monitoring workers in isolated locations.
 4. Lack of centralized sensor data.
 5. Lack of historical safety records.
 6. Difficulty tracking worker location during emergencies.
 7. Manual monitoring limitations.
 8. Lack of real-time dashboard visualization.
-

3. Proposed Solution

The proposed system connects the worker monitoring hardware to a cloud platform using the **ESP8266 NodeMCU's Wi-Fi capability**.

The sensors collect information from the worker and workplace. The ESP8266 processes the readings and determines whether the system is operating under normal or warning conditions.

The processed information is transmitted to a cloud platform such as **ThingSpeak, Blynk, Firebase, or an MQTT-based server**.

The cloud platform stores the data and displays it on a dashboard.

4. System Architecture

The system architecture consists of six main layers.

Layer 1 – Sensing Layer

The sensing layer collects information from:

- Pulse Sensor
- DHT11
- MQ-2
- NEO-6M GPS

Layer 2 – Processing Layer

The **ESP8266 NodeMCU** processes the sensor readings and determines the current system status.

Layer 3 – Local Monitoring Layer

The OLED display shows sensor values and system status. The buzzer provides an immediate local warning when an unsafe condition is detected.

Layer 4 – Communication Layer

The ESP8266 connects to the Internet through Wi-Fi.

Layer 5 – Cloud Layer

The cloud platform receives, stores, and processes sensor information.

Layer 6 – User Layer

The supervisor accesses the information through a web or mobile dashboard.

5. Circuit Table

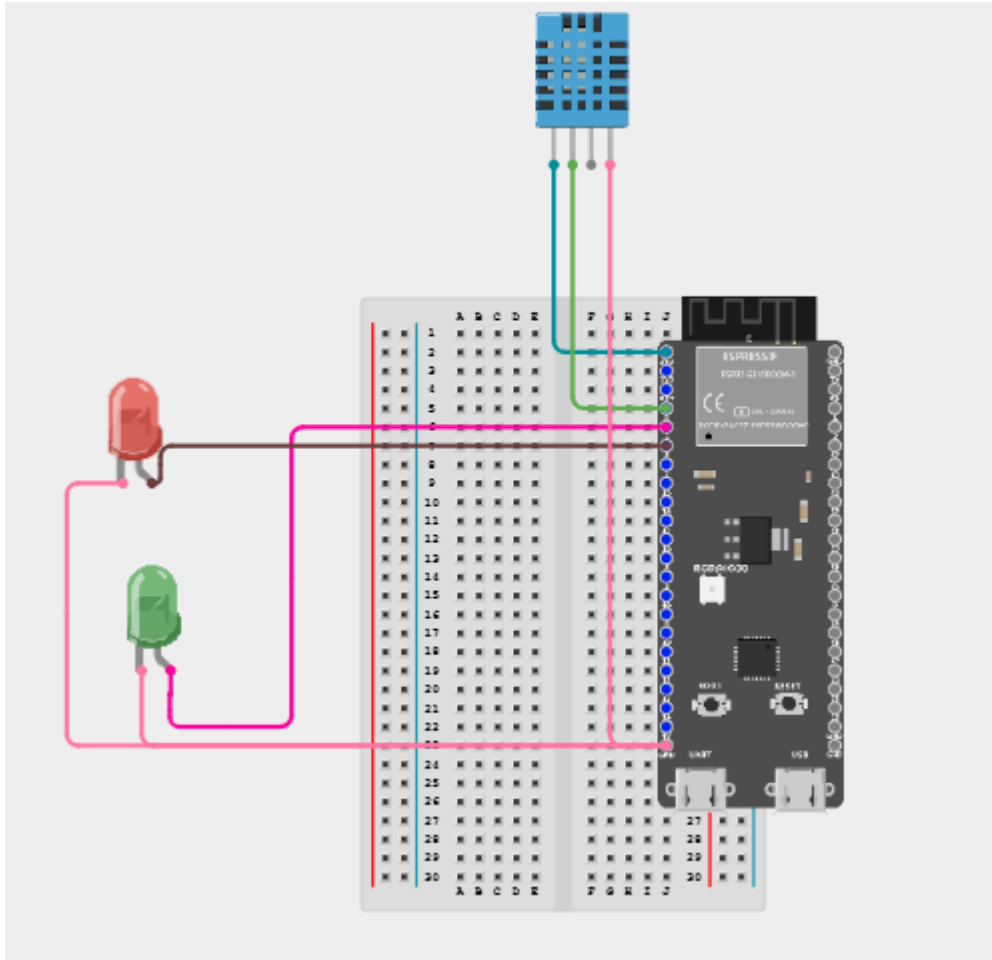
Component	Pin	ESP8266 NodeMCU	Function
Pulse Sensor	VCC	3.3V	Power
Pulse Sensor	GND	GND	Ground
Pulse Sensor	Signal	A0	Pulse signal
DHT11	VCC	3.3V	Power
DHT11	GND	GND	Ground
DHT11	DATA	D3 / GPIO0	Temperature/Humidity
MQ-2	VCC	5V/VU	Power
MQ-2	GND	GND	Ground
MQ-2	DO	D0 / GPIO16	Gas status
Buzzer	+	D7 / GPIO13	Alarm
Buzzer	-	GND	Ground
OLED	VCC	3.3V	Power
OLED	GND	GND	Ground
OLED	SDA	D2 / GPIO4	I ² C data
OLED	SCL	D1 / GPIO5	I ² C clock
GPS	VCC	Suitable supply	Power
GPS	GND	GND	Ground
GPS	TX	D5 / GPIO14	GPS data
GPS	RX	D6 / GPIO12	GPS commands

Important note

The ESP8266 has only **one analog input, A0**. Therefore, the Pulse Sensor uses A0, while the MQ-2 is connected through its **digital output (DO)** in this prototype.

If the MQ-2 module's digital output can exceed the ESP8266 GPIO voltage limit, use a **voltage divider or suitable level shifter**.

6. Circuit Design



The ESP8266 is the central controller. All sensors provide data to the controller, and the controller provides outputs to the OLED, buzzer, and cloud platform.

Pulse Sensor

Pulse Sensor ESP8266

VCC 3.3V

GND GND

SIGNAL A0

The Pulse Sensor provides an analog waveform that can be processed to estimate pulse rate.

DHT11

DHT11 ESP8266

VCC ————— 3.3V
GND ————— GND
DATA ————— D3

The DHT11 measures temperature and relative humidity.

MQ-2

MQ-2 ESP8266

VCC ————— 5V/VU
GND ————— GND
DO — Level Shift — D0

The digital output is used to determine whether the configured gas threshold has been crossed.

OLED

OLED ESP8266

VCC ————— 3.3V
GND ————— GND
SDA ————— D2
SCL ————— D1

The OLED communicates using I²C.

GPS

NEO-6M GPS ESP8266

VCC ————— Suitable supply
GND ————— GND
TX ————— D5
RX ————— D6

The GPS module sends NMEA location data to the ESP8266.

Buzzer

ESP8266 D7 ————— Buzzer +

GND ————— Buzzer -

A transistor driver should be used if the buzzer requires more current than the GPIO can safely provide.

7. Algorithm

Cloud monitoring algorithm

Step 1: Start the system.

Step 2: Power ON the ESP8266.

Step 3: Initialize all sensors.

Step 4: Initialize OLED and GPS communication.

Step 5: Connect ESP8266 to Wi-Fi.

Step 6: Read Pulse Sensor data.

Step 7: Read temperature and humidity from DHT11.

Step 8: Read gas status from MQ-2.

Step 9: Read GPS data.

Step 10: Process the sensor information.

Step 11: Display sensor values on the OLED.

Step 12: Compare sensor readings with configured safety conditions.

Step 13: If an unsafe condition is detected:

- Activate buzzer.
- Display warning.
- Set emergency status.
- Upload alert to cloud.

Step 14: If conditions are normal:

- Display SAFE.
- Upload normal readings.

Step 15: Store the readings in the cloud.

Step 16: Update the remote dashboard.

Step 17: Repeat continuously.

9. Coding

```
#include <ESP8266WiFi.h>
#include <DHT.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"

// =====
// WIFI SETTINGS
// =====

#define WIFI_SSID  "Pradeep"
#define WIFI_PASSWORD "12345678"

// =====
// ADAFRUIT IO SETTINGS
// =====

#define AIO_SERVER  "io.adafruit.com"
#define AIO_SERVERPORT 1883

#define AIO_USERNAME  "muthumkt"
#define AIO_KEY       "aio_Dpts45gXpVu4he3HHcwUXVBgOuKb"

// =====
// DHT11 SENSOR
// =====

#define DHT_PIN  D7
#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

// =====
// WIFI CLIENT
// =====

WiFiClient client;
```

```

// =====
// ADAFRUIT MQTT
// =====

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// =====
// ADAFRUIT IO FEEDS
// =====

Adafruit_MQTT_Publish temperatureFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/temperature"
  );

Adafruit_MQTT_Publish humidityFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/humidity"
  );

// =====
// WIFI CONNECTION
// =====

void connectWiFi()
{
  if (WiFi.status() == WL_CONNECTED)
  {
    return;
  }

  Serial.println();
  Serial.print("Connecting to WiFi: ");
  Serial.println(WIFI_SSID);

```

```

WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

while (WiFi.status() != WL_CONNECTED)
{
    delay(500);
    Serial.print(".");
}

Serial.println();
Serial.println("WiFi Connected!");

Serial.print("IP Address: ");
Serial.println(WiFi.localIP());
}

// =====
// ADAFRUIT IO CONNECTION
// =====

void connectMQTT()
{
    if (mqtt.connected())
    {
        return;
    }

    Serial.println("Connecting to Adafruit IO...");

    int8_t ret;

    while ((ret = mqtt.connect()) != 0)
    {
        Serial.print("MQTT Error: ");
        Serial.println(mqtt.connectErrorString(ret));

        mqtt.disconnect();

        delay(5000);
    }

    Serial.println("Adafruit IO Connected!");
}

```

```

// =====
// SETUP
// =====

void setup()
{
  Serial.begin(115200);

  dht.begin();

  delay(2000);

  connectWiFi();
  connectMQTT();

  Serial.println();
  Serial.println("=====");
  Serial.println(" CLOUD INTEGRATION & REMOTE MONITORING");
  Serial.println("=====");
  Serial.println("DHT11      : READY");
  Serial.println("WiFi      : CONNECTED");
  Serial.println("Adafruit IO : CONNECTED");
  Serial.println("System    : READY");
  Serial.println();
}

// =====
// MAIN LOOP
// =====

void loop()
{
  // Check WiFi
  if (WiFi.status() != WL_CONNECTED)
  {
    connectWiFi();
  }

  // Check Adafruit IO
  connectMQTT();

  // =====

```

```

// READ DHT11
// =====

float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

// =====
// SENSOR ERROR
// =====

if (isnan(temperature) || isnan(humidity))
{
    Serial.println("DHT11 SENSOR ERROR!");
    delay(2000);
    return;
}

// =====
// DISPLAY DATA
// =====

Serial.println("-----");

Serial.print("Temperature : ");
Serial.print(temperature, 1);
Serial.println(" C");

Serial.print("Humidity   : ");
Serial.print(humidity, 1);
Serial.println(" %");

// =====
// SEND TEMPERATURE TO CLOUD
// =====

if (temperatureFeed.publish(temperature))
{
    Serial.println("Temperature : CLOUD UPDATED");
}
else
{
    Serial.println("Temperature : UPLOAD FAILED");
}

```

```

// =====
// SEND HUMIDITY TO CLOUD
// =====

if (humidityFeed.publish(humidity))
{
  Serial.println("Humidity  : CLOUD UPDATED");
}
else
{
  Serial.println("Humidity  : UPLOAD FAILED");
}

Serial.println("-----");
Serial.println();

// Update every 10 seconds
delay(10000);
}

```

9. System Working

The system starts when power is supplied to the ESP8266 NodeMCU.

First, the ESP8266 initializes all sensors and connects to the configured Wi-Fi network.

The **Pulse Sensor** collects the worker's pulse waveform. The ESP8266 processes the signal to obtain useful pulse information.

The **DHT11** measures temperature and humidity. These readings are useful for monitoring environmental conditions.

The **MQ-2** monitors gas/smoke conditions. If its configured threshold is exceeded, the system changes the status to WARNING or EMERGENCY.

The **NEO-6M GPS** receives satellite information and provides the worker's geographical coordinates when a valid fix is available.

The OLED continuously displays the current system information.

At the same time, the ESP8266 sends the data to the cloud through Wi-Fi.

The supervisor can therefore monitor the worker remotely and identify safety events.

10. Bill of Materials

S.No.	Component	Quantity	Approx. Cost
1	ESP8266 NodeMCU	1	₹300–₹500
2	Pulse Sensor	1	₹150–₹300
3	DHT11 Sensor	1	₹40–₹100
4	MQ-2 Gas Sensor	1	₹80–₹150
5	Buzzer	1	₹10–₹30
6	OLED Display 0.96"	1	₹150–₹300
7	NEO-6M GPS Module	1	₹400–₹700
8	Breadboard	1	₹80–₹150
9	Jumper Wires	1 set	₹50–₹150
10	Resistors/Voltage Divider	1 set	₹20–₹50
11	USB Cable	1	₹50–₹150

Estimated cost

The approximate prototype cost is **₹1,500–₹3,000**, depending on component quality, supplier, power source, and cloud-service requirements.

11. Prototype Implementation

The prototype is implemented by integrating all hardware components with the ESP8266 Node MCU.

Step 1 – Hardware assembly

The ESP8266 is placed on a breadboard and the power and ground connections are established.

Step 2 – Sensor integration

The Pulse Sensor is connected to A0, DHT11 to D3, and MQ-2 digital output to D0 through an appropriate voltage interface.

Step 3 – OLED integration

The OLED is connected through the I²C interface.

Step 4 – GPS integration

The NEO-6M GPS module is connected through serial communication.

Step 5 – Buzzer integration

The buzzer is connected to the configured digital output and is activated when an unsafe condition occurs.

Step 6 – Software implementation

The ESP8266 is programmed using Arduino IDE. The program initializes the sensors, reads the data, processes the information, and manages the Wi-Fi connection.

Step 7 – Cloud integration

The ESP8266 sends sensor readings to the selected cloud platform using an Internet connection.

Step 8 – Dashboard development

A dashboard is configured to display:

- Worker ID
- Pulse information
- Temperature
- Humidity
- Gas state

12. Results & Performance Analysis

The prototype is tested by checking each sensor independently and then evaluating the complete cloud-connected system.

Expected results

Parameter	Expected Result	System Action
Pulse	Pulse signal received	Display/upload data
Temperature	DHT11 reading obtained	Display/upload data
Humidity	DHT11 reading obtained	Display/upload data
Gas	Safe/Danger status	Buzzer when alarm occurs

Parameter	Expected Result	System Action
GPS	Latitude/Longitude	Upload to cloud
OLED	Sensor information	Local monitoring

Performance analysis

1. Data collection

The ESP8266 can periodically collect information from the connected sensors and prepare it for cloud transmission.

2. Cloud communication

The built-in Wi-Fi capability of the ESP8266 allows the device to communicate with an Internet-connected cloud platform.

3. Remote monitoring

Sensor information can be viewed from a web or mobile dashboard, reducing the need for continuous physical supervision.

4. Alert response

When a configured unsafe condition is detected, the local buzzer provides immediate feedback. The system can also transmit an alert to the cloud.

5. GPS monitoring

The NEO-6M can provide geographical coordinates when it has a valid satellite fix. This information can be used to identify the approximate location of the worker.

6. Data storage

Cloud storage allows previous readings and safety events to be retained for later analysis.

7. Scalability

The system can be expanded to support multiple workers by assigning each monitoring device a unique worker/device ID.

8. Reliability considerations

The system's remote-monitoring performance depends on:

- Wi-Fi availability
- Internet connectivity
- Cloud-server availability
- Sensor accuracy

- Battery/power availability
- GPS signal quality

Performance table

Performance Parameter	Prototype Observation
Sensor data acquisition	Continuous/periodic
Local display	Real-time/periodic
Gas warning	Immediate local alarm after threshold detection
Cloud transmission	Dependent on Wi-Fi
GPS	Dependent on satellite reception
Data storage	Cloud dependent
Remote accessibility	Available when Internet is available

Conclusion

The **Cloud Integration and Remote Monitoring System** provides a complete IoT framework for remotely monitoring worker safety. The ESP8266 Node MCU acts as the central processing and communication unit, collecting information from the Pulse Sensor, DHT11, MQ-2, and NEO-6M GPS.

The collected information is displayed locally through the OLED and safety conditions are indicated through the buzzer. Using Wi-Fi, the ESP8266 can transmit the data to a cloud platform where it can be stored, visualized, and monitored remotely.

The proposed system can significantly improve the **visibility of worker safety information, emergency awareness, and remote supervision**. It can be further enhanced by adding fall detection, emergency buttons, mobile notifications, geofencing, multiple-worker support, advanced cloud analytics, and AI-based safety prediction.

Output

The top image shows an Arduino IDE window with the following code:

```
while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
}  
  
Serial.println();  
Serial.println("WiFi Connected!");  
  
Serial.print("IP Address: ");  
Serial.println(WiFi.localIP());  
}  
  
// =====  
// CONNECT TO ADAFRUIT IO  
// =====  
  
void connectMQTT() {  
    if (mqtt.connected()) {  
        return;  
    }  
  
    Serial.print("Connecting to Adafruit IO...");  
  
    int8_t ret;  
  
    while ((ret = mqtt.connect()) != 0) {  
        Serial.println(mqtt.connectErrorString(ret));  
        mqtt.disconnect();  
    }  
}
```

The serial monitor window (COM7) displays the following output:

```
Humidity: 72.90 %  
LED: OFF  
STATUS: NORMAL  
Temperature uploaded successfully  
Humidity uploaded successfully  
=====  
Temperature: 29.10 °C  
Humidity: 72.70 %  
LED: OFF  
STATUS: NORMAL  
Temperature uploaded successfully  
Humidity uploaded successfully  
=====
```

The bottom image shows the Adafruit IO dashboard for the user 'muthumkt'. The dashboard includes a 'Monitor' tab with a live data feed and a 'Value' gauge showing 82. The dashboard also features a sidebar with links to 'What is Adafruit IO?', 'Get Help', 'Quick Guides', 'API Documentation', 'FAQ', 'Freebies', 'Terms of Service', 'Privacy Policy', 'Website Accessibility', and 'Send Feedback'. The main content area displays the following text:

leaving...
Hard resetting via RTS pin...

Shop Learn Blog Forums IO LIVE! AdaBox Account 0

adafruit Devices Feeds Dashboards Actions Power-Ups New Device

muthumkt / Dashboards / Monitor New Block Edit Layout

temperature 29.10
2026/08/20 09:31:13AM Default
temperature 29.20
2026/08/20 09:31:13AM Default
temperature 29.20
2026/08/20 09:31:13AM Default
temperature 30.30
2026/08/20 09:31:13AM Default
temperature 30.30
2026/08/20 09:31:13AM Default
temperature 30.50
2026/08/20 09:31:13AM Default
temperature 30.50
2026/08/20 09:31:13AM Default

82 Value

IO Status Learn IO Plus News

"Meaning can easily be confused with structure.
Structure is a good approximation, but not the same"
— Claudia Perlich